

AmanahCX Developer & Integration Partner Orientation

This document orients full-stack engineering partners to the AmanahCX platform: what it is, how its pieces fit together, and how to connect Nadia (the voice agent), the mobile app, or a customer's other systems (CRM, softphone, or automation tools) to it. Read section 2 first for the big picture, then jump to whichever component you're integrating with.

Index

1. What AmanahCX Is

1.1 End-to-End Information Flow

2. System Landscape

2.1 Components Diagram

3. CRM Platform Architecture

3.1 Architecture Diagram

3.2 Multi-Tenancy & Security

3.3 Access Control Model

3.4 Business Modules

4. Nadia Voice Agent Architecture

4.1 SIP + Voice Pipeline Diagram

4.2 The Call Pipeline, Step by Step

4.3 The CRM Connector Contract

4.4 Connecting Nadia to a Different CRM

5. Mobile App Architecture

5.1 Architecture Diagram

5.2 Screens & Functionality

5.3 Offline & On-Device AI

6. Connecting Other Applications

6.1 REST & GraphQL API

6.2 Outbound Webhooks

6.3 API Keys (Inbound)

6.4 Telephony / Softphone Integration

6.5 Common Integration Patterns

7. Data Model & Auth Essentials

8. Environments & Deployment

9. Consolidated Build & Release Report

9.1 Three Pipelines, One Repo

9.2 CRM Web + API Builds

9.3 Nadia Voice Agent Builds

9.4 Mobile App Builds

10. Self-Hosted Migration (VM + Domain)

10.1 Target Architecture Diagram

10.2 Migration Runbook

10.3 What You Gain, What You Now Own

11. Repository Map

12. Glossary

1. What AmanahCX Is

AmanahCX is a **multi-tenant SaaS platform** that combines a CRM, a contact-centre (with an AI voice agent), sales & invoicing, analytics, and a field-staff mobile app — one system hosting many isolated customer organisations ("tenants"). A typical tenant is a bank, an insurer, or a services business that wants: customers to call in and get handled by an AI voice agent in Urdu/English, every enquiry to become a tracked ticket with an SLA clock, and their field/sales staff to work from a phone app.

As an integration partner, your work generally falls into one (or more) of three buckets:

- **Connect Nadia (voice) to a different CRM** — a customer already has their own CRM (Salesforce, HubSpot, Dynamics, a bespoke system) and wants Nadia's voice capability without adopting AmanahCX's CRM. See Section 4.
- **Connect another application to AmanahCX** — a softphone, a BI tool, an automation platform (Zapier-style), or a customer's existing systems need to read/write AmanahCX data or receive events from it. See Section 6.
- **Extend or customise AmanahCX itself** for a specific customer deployment — new module, new field, new workflow. See Section 3 and Section 11.

AmanahCX is **multi-tenant and multi-sector**: every customer organisation is a fully isolated tenant (§3.2), and each tenant is provisioned against a sector template (banking, insurance, education, and others) — creating a tenant automatically populates the sector-specific fields its contacts, tickets, and deals need (e.g. Case Type / Transaction Reference / Regulatory Deadline for a banking tenant), with no manual field setup required (§4.17 of the Master Product Document). Every label in the system is a starting point, not a fixed name: a tenant admin can rename any role or responsibility title (e.g. "Line Manager" → "Territory Manager") and any field name to match how that organisation actually talks about its own work — the underlying permission model and data structure never change, only what they're called.

1.1 End-to-End Information Flow

The diagram below traces one enquiry from the moment a customer dials in, through Nadia's contact matching and ticket creation, ownership and routing, SLA/TAT governance, resolution and escalation, to completion and CSAT — the single flow every integration in this document eventually plugs into somewhere.

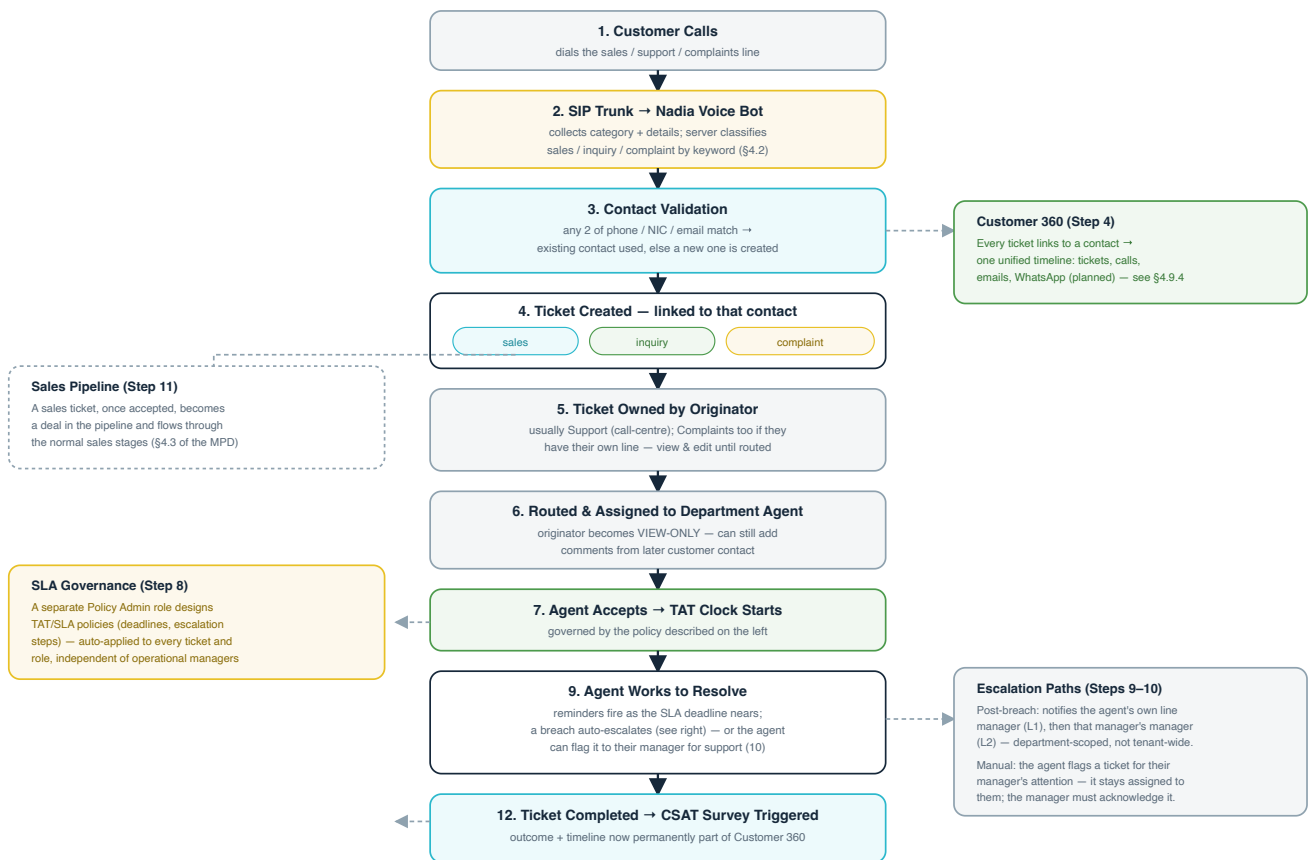


Fig 1.1 — one enquiry's full lifecycle: intake through Nadia, contact matching, ticket ownership and routing, SLA-governed resolution, and completion.

This flow was checked directly against the code (not just prior documentation) on 2026-07-18. Two real bugs were found and fixed as part of that check:

1. In `createComplaintFromStructured` (`packages/api/src/routes/voice-bot.ts`), a Nadia-created ticket's `ticket_type` column was hardcoded to the literal `'complaint'` regardless of the sales/inquiry/complaint classification computed for queue routing — so a sales enquiry captured by voice routed to the right queue but could never satisfy the `ticket_type === 'sales'` check that creates a pipeline deal on accept (§4.3). Fixed to persist the actual computed type.
2. SLA breach escalation (`runSlaWorker` in `tickets.ts`) notified **every** manager/tenant_admin tenant-wide on breach, regardless of department — a Support ticket breach would also page Sales and Complaints managers. This broke the department-isolation model the rest of the system enforces via the reporting tree (`lib/visibility.ts`). Fixed with a new `getEscalationTargets()` helper: L1 now notifies only the assignee's own line manager, L2 only that manager's manager, walking `users.manager_id` the same way visibility checks already do elsewhere — falling back to tenant_admin only when an agent has no manager configured, so a breach is never silently unnoticed.

Two step descriptions above were also corrected to match real behavior: escalation (steps 9–10) now correctly reflects the assignee's own two-level manager chain, department-scoped, not a tenant-wide broadcast; contact matching (step 3) accepts any two of phone/NIC/email, not only pairs that include phone.

2. System Landscape

At the highest level, AmanahCX is **three independently-deployable pieces** talking over HTTP(S)/SIP — not one monolith. This separation is exactly what makes each piece reusable outside AmanahCX: the CRM API doesn't know or care whether a request came from the web app, the mobile app, Nadia, or a third-party integration — they all speak the same REST/GraphQL contract.

Web Platform

CRM Web App + API

React frontend + Fastify API + PostgreSQL. The system of record: contacts, deals, tickets, sales, roles, settings. Details →

Voice

Nadia Voice Agent

Independent Python service on LiveKit. Takes phone calls over SIP, talks to whichever CRM is configured via a small connector contract. Details →

Mobile

Field Staff App

Expo/React Native app for field sales & support staff. Talks to the same CRM API as the web app. Details →

2.1 Components Diagram

CLIENTS

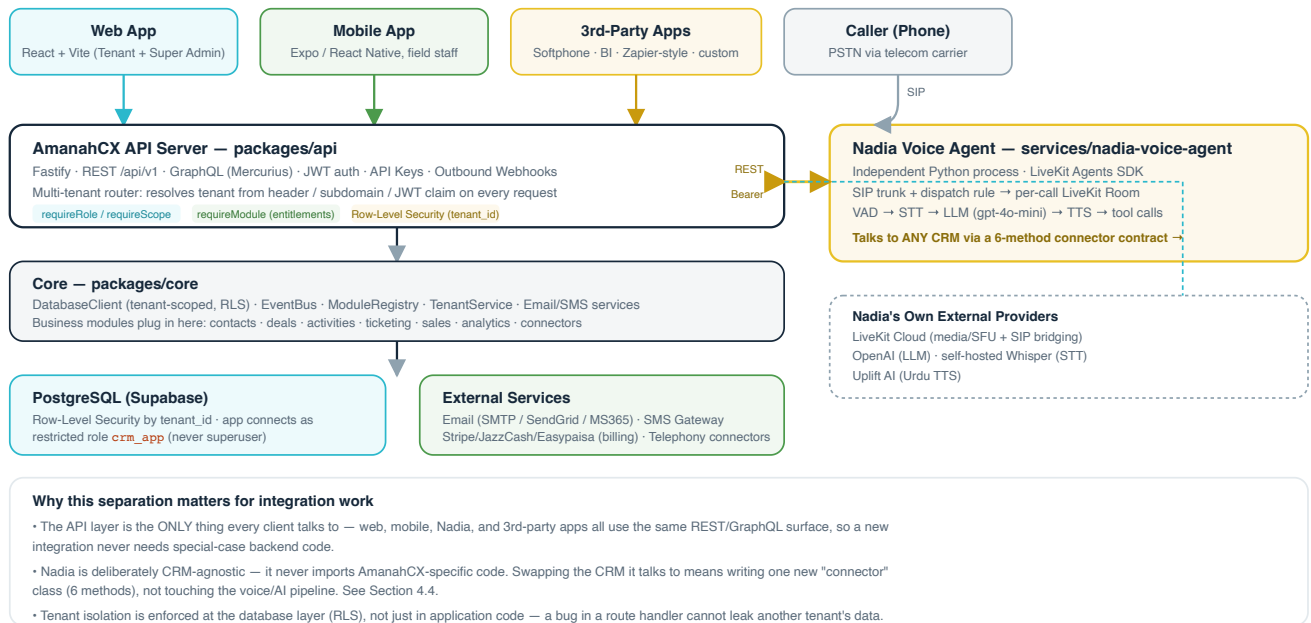


Fig 2.1 — AmanahCX component landscape: three independently-deployable services sharing one API contract.

2.2 Technology Stack by Component

Every part of the system is built in the language best suited to what it does — there is no single "one language" answer. Knowing this up front tells you what to expect when you open each folder.

COMPONENT	LANGUAGE	RUNTIME / FRAMEWORK	KEY LIBRARIES	BUILD TOOL
Frontend packages/frontend	TypeScript	React 18.3 (SPA)	TanStack Query 5 + TanStack Table, Tailwind CSS 3	Vite 5
API packages/api	TypeScript	Node.js + Fastify 4	GraphQL 16 + Mercurius 14, Zod 3 (validation), jsonwebtoken + bcryptjs (auth)	tsc → dist/
Core packages/core	TypeScript	Node.js library (no server of its own)	<code>pg</code> (node-postgres, tenant-scoped client), <code>ioredis</code>	tsc
Shared packages/shared	TypeScript	—	Cross-package types/contracts, sector config	tsc
Business Modules modules/*	TypeScript	Plug into Core's ModuleRegistry	Twilio/Vonage SDKs (telephony connectors)	tsc

Mobile App packages/mobile	TypeScript	React Native 0.74 on Expo SDK 51	Expo Router, Zustand, TanStack Query, ML Kit (on-device OCR), expo-speech-recognition	Metro (dev) / EAS Build (release)
Nadia Voice Agent services/nadia-voice-agent	Python 3.11	LiveKit Agents SDK 1.5	faster-whisper (self-hosted STT), httpx (async HTTP to CRM), livekit-plugins-{openai, upliftai, silero, turn-detector}	pip / venv (Docker image in production — see §9.3)
Database	SQL (PostgreSQL 16 dialect)	PostgreSQL 16	Row-Level Security policies · 66 ordered migration files (<code>packages/core/src/database/migrations</code>)	plain .sql, applied in order
Monorepo tooling	—	—	Turborepo (build orchestration) + npm workspaces	—

Every TypeScript package shares one root `tsconfig` lineage (ES2022 target, strict mode, CommonJS output) — Node never runs the `.ts` files directly, it runs the compiled `dist/`. Python is entirely isolated to Nadia: nothing else in the platform touches it, and Nadia touches nothing else in the platform except through its CRM connector's HTTP calls (§4.3). If you're a full-stack developer coming from either a Node or Python background, you can be immediately productive in your own language on either Nadia or everything else, without needing the other.

3. CRM Platform Architecture

Stack

Frontend	React + TanStack Query, Vite — <code>packages/frontend</code>
API	Fastify, REST (<code>/api/v1</code>) + GraphQL (Mercurius), Zod validation, JWT auth — <code>packages/api</code>
Core	Tenant-aware DB client, EventBus, ModuleRegistry, TenantService — <code>packages/core</code>
Database	PostgreSQL with Row-Level Security, hosted on Supabase, 45+ ordered migrations
Modules	Pluggable business capabilities — <code>modules/*</code> (contacts, deals, activities, ticketing, sales, analytics, connectors)

Repo layout at a glance

```
solution/
├── packages/
│   ├── frontend/    # React SPA (tenant + super admin views)
│   ├── api/         # Fastify REST + GraphQL server
│   ├── core/        # DB client, EventBus, ModuleRegistry, TenantService
│   ├── shared/      # Types/contracts shared across packages
│   └── mobile/      # Expo/React Native field app
├── modules/
│   ├── contacts/ deals/ activities/ ticketing/
│   ├── sales/ analytics/ voice/ voice-module/
│   └── connectors/  # Twilio/Vonage telephony adapters
├── services/
│   └── nadia-voice-agent/ # independent Python voice service
└── docs/            # this document + product/ops/change docs
```

3.1 Architecture Diagram

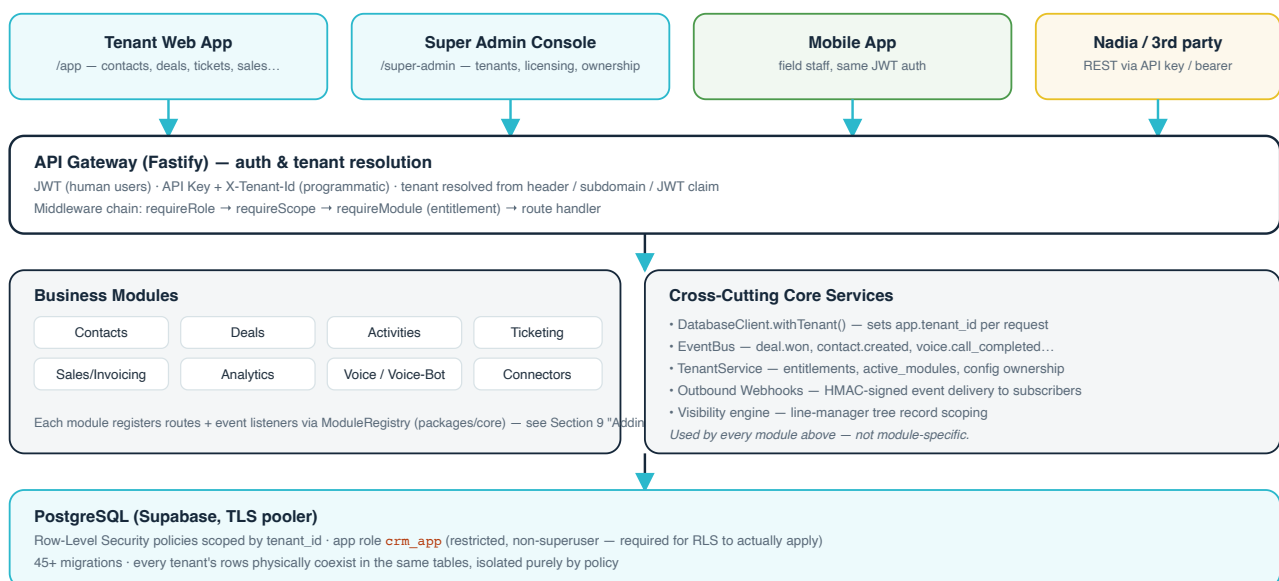


Fig 3.1 — CRM platform layers: clients → API gateway → business modules + core services → database.

3.2 Multi-Tenancy & Security

- **Row-Level Security (RLS):** every tenant's data lives in the same tables, isolated by a `tenant_id` column and Postgres RLS policies. The API sets `app.tenant_id` as a session variable at the start of each transaction (`DatabaseClient.withTenant`).
- **Non-superuser DB role is mandatory:** the app connects as a restricted role (`crm_app`), never as `postgres` . A `BYPASSRLS` or `SUPERUSER` role silently ignores every RLS policy — this was a real production incident, fixed 2026-07-10. Any integration that touches `DATABASE_URL` must preserve this.
- **Tenant resolution** on every request, in priority order: `X-Tenant-Id` header → subdomain → custom domain → JWT claim.

3.3 Access Control Model

Four layers, each answering a different question:

LAYER	QUESTION IT ANSWERS	MECHANISM
1. Entitlement	What has the tenant licensed?	<code>tenants.entitled_features</code> / <code>active_modules</code> — set by Super Admin, checked via <code>requireModule()</code>
2. Roles	What can this role do?	Tenant admin builds roles (Admin/Manager/Agent/Viewer + custom + Policy Admin governance role) — checked via <code>requireRole()</code> / <code>requireScope()</code>
3. Record visibility	Whose records can this user see?	Line-manager tree filter — agent sees own, line manager sees team, manager sees department
4. Separation of duties	Structural guardrails	e.g. <code>tenant_admin</code> is administrative-only, blocked from operational/billing routes

For any new integration, decide up front which of these four layers your integration's credentials should sit at — e.g. a softphone integration typically needs a scoped API key (layer 2), not a full admin JWT.

3.4 Business Modules

Core CRM (contacts, companies, deals, activities) · Sales & Invoicing (invoices, quotations, payments, templates) · Ticketing/Contact-Centre (queues, SLA/TAT policies, CSAT, escalation) · Voice + Voice-Bot (Nadia) · Email inbox · Analytics & Reports · Team Messaging · Integrations (connectors, webhooks, API keys) · Departments & Opportunities · Super-Admin console.

4. Nadia Voice Agent Architecture

Nadia is a **self-hosted alternative to Retell AI / Vapi**, purpose-built for the Pakistani market (Urdu/English, self-hosted STT, local TTS voice). It runs as its own Python service — `services/nadia-voice-agent` — completely independent of the AmanahCX codebase. It is deliberately CRM-agnostic: it never imports AmanahCX-specific code.

4.1 SIP + Voice Pipeline Diagram

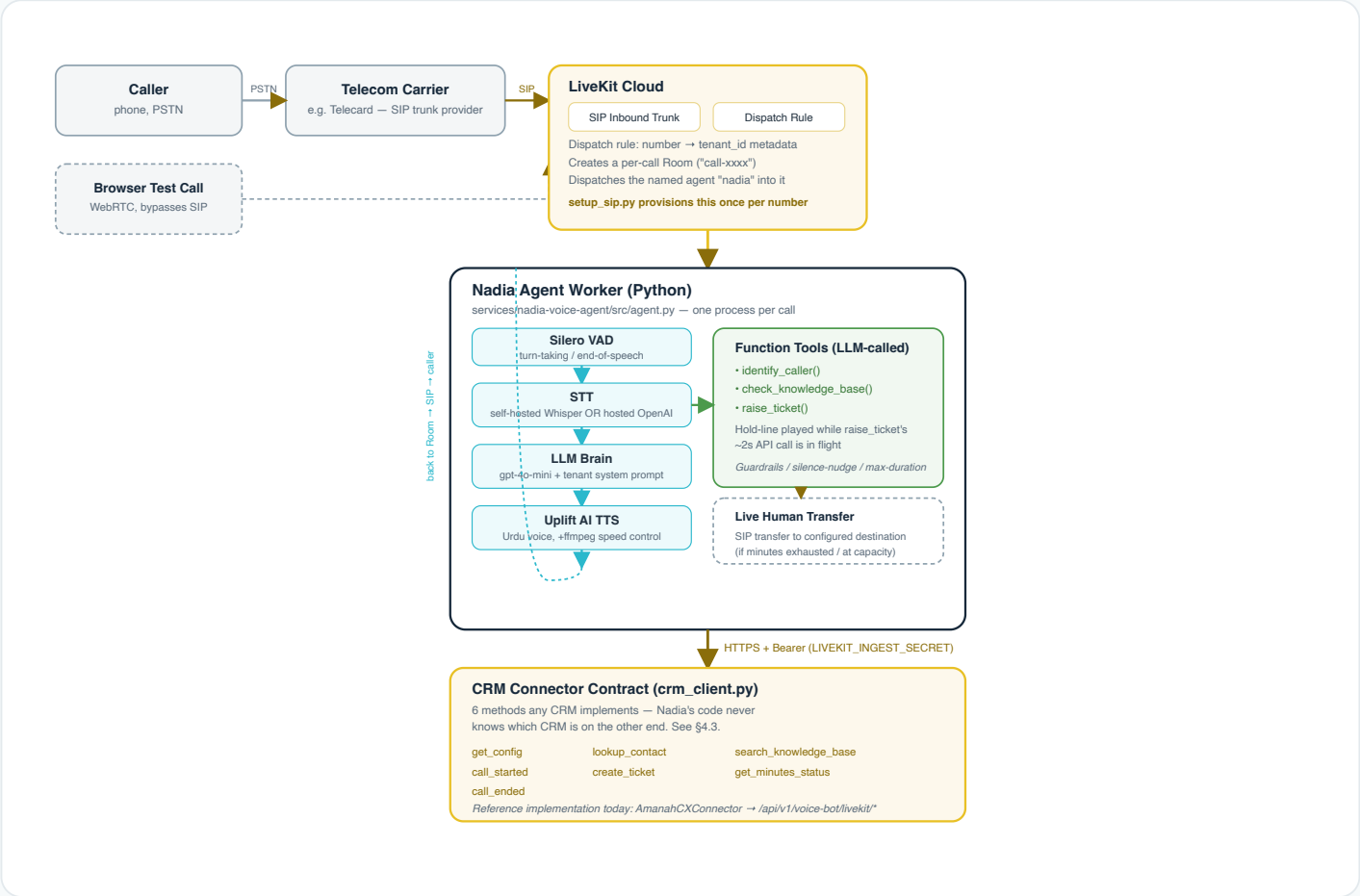


Fig 4.1 — Nadia's SIP-to-speech pipeline and its connector boundary to whichever CRM is configured.

4.2 The Call Pipeline, Step by Step

- 1 **SIP wiring (one-time, per phone number):** `setup_sip.py` creates a LiveKit inbound SIP trunk for the carrier's number, plus a dispatch rule that puts every inbound call into its own room and dispatches the named agent `nadia` into it, carrying the tenant ID as room metadata.
- 2 **Call arrives:** caller dials the number → carrier delivers it over SIP to LiveKit Cloud → dispatch rule creates room `call-xxxx` and starts a Nadia worker process for it.
- 3 **Settings load:** the worker extracts the tenant ID from job metadata and calls `get_config()` on the CRM connector to fetch that tenant's bot name, tone, speaking rate, voice, system prompt, and guardrails — this is what makes Nadia's behaviour fully configurable per-tenant from the CRM's own admin screen, no redeploy needed.
- 4 **Speech loop:** Silero VAD detects when the caller stops talking → buffered audio goes to STT (self-hosted Whisper on capable hardware, or hosted OpenAI transcription on constrained hosts) → transcript goes to the gpt-4o-mini LLM with the tenant's system prompt → LLM either replies directly or calls a tool.
- 5 **Tools:** `identify_caller` (recognise a returning contact), `check_knowledge_base` (answer general questions without a ticket), `raise_ticket` (file a complaint/case — plays a spoken hold line while the ~2s API call is in flight).
- 6 **Reply spoken back:** LLM text → Uplift AI TTS → (optional ffmpeg time-stretch if speaking rate ≠ 1.0) → audio frames → back down the same room/SIP leg to the caller.
- 7 **Guardrails running throughout:** concurrency/capacity gate and minutes-allocation gate (checked right after connect — if exhausted, Nadia hands off with an urgent ticket + optional live SIP transfer instead of running normally); silence nudge (asks twice, then ends the call); hard max-call-duration cutoff.
- 8 **Call ends:** full transcript (+ recording URL if enabled) is reported back to the CRM via `call_ended()`.

4.3 The CRM Connector Contract

This is the single most important thing to understand for integration work: **Nadia's voice/AI code never talks to a specific CRM's API directly.** Every CRM-facing call goes through one Python `Protocol` with exactly six methods, defined in `services/nadia-voice-agent/src/crm_client.py` :

```
class CRMConnector(Protocol):
    async def get_config(tenant_id) -> dict | None
    async def create_ticket(tenant_id, call_id, ticket: dict) -> dict
    async def search_knowledge_base(tenant_id, question) -> dict | None
    async def lookup_contact(tenant_id, phone, nic, email) -> dict
    async def call_started(tenant_id, call_id) -> dict          # concurrency/capacity check
    async def get_minutes_status(tenant_id) -> dict             # minutes-allocation check
    async def call_ended(tenant_id, call_id, transcript, recording_url) -> None
```

Today there is exactly one implementation, `AmanahCXConnector` , which calls AmanahCX's own REST endpoints under `/api/v1/voice-bot/livekit/*` with a bearer token (`LIVEKIT_INGEST_SECRET`). `get_client()` picks the active connector from the `CRM_CONNECTOR` env var.

4.4 Connecting Nadia to a Different CRM

To connect Nadia to Salesforce, HubSpot, Dynamics, or a bespoke CRM: write one new class implementing the six `CRMConnector` methods against that CRM's own API/webhooks, and register it in `get_client()` . Nothing in `agent.py` , the VAD/STT/LLM/TTS pipeline, the SIP setup, or the tool definitions needs to change.

METHOD	WHAT THE TARGET CRM NEEDS TO EXPOSE
<code>get_config</code>	Return the tenant's bot settings (name, tone, voice, system prompt, guardrails). If the target CRM has no such concept yet, this can start as a static/env-driven config.
<code>create_ticket</code>	Create a case/ticket/lead record and return an ID Nadia can read back to the caller.
<code>search_knowledge_base</code>	Optional — return a direct-answer match for general questions. Can return <code>None</code> always if the target CRM has no KB.
<code>lookup_contact</code>	Search by phone/NIC/email, return a confidence-graded match so Nadia can (or cannot) greet a returning caller by name.
<code>call_started</code> / <code>get_minutes_status</code>	Optional gating hooks — can be stubbed to always return "not exhausted / not over capacity" if the target deployment doesn't need usage limits.
<code>call_ended</code>	Persist the transcript/recording somewhere on the target CRM's side (case notes, call log, etc.).

Everything else — the SIP trunk, LiveKit dispatch rule, VAD/STT/LLM/TTS pipeline, guardrails, silence handling, live transfer — is entirely CRM-independent and is reused as-is.

Nadia currently lives inside the AmanahCX repo as a convenience, but architecturally it is already a standalone product (comparable to Retell AI / Vapi) — the connector pattern above is exactly what makes that possible without a rewrite.

4.5 Agent Builder — Reusable Templates & Auto-Provisioning (2026-07-18)

Before this, every tenant's bot was configured by hand-editing that tenant's `voice_bot_configs` row directly. Three additions make this reusable and (for licensed tenants) automatic:

- Agent Templates** (`voice_bot_agent_templates` , Super Admin → Agent Templates) — a reusable "recipe" (name, sector, company name, department, tone, character, language, guardrails, voice, call direction, bot engine) created once and assigned to any workspace. Assigning is a *one-time copy* into that tenant's `voice_bot_configs` row — not a live binding — so it stays independently editable afterward. Assigning also

auto-grants the `voice_bot` module (+ the LiveKit provider feature) if the tenant wasn't already licensed for it.

- **Multi-source knowledge base at the Super Admin level** — the same 3-way ingestion the tenant-side page already had (typed text, URL crawl-and-extract, PDF/DOCX upload with server-side text extraction) is now also available when Super Admin manages a tenant's KB directly.
- **Auto-provisioning on tenant creation** — in `POST /super-admin/tenants`, if the new tenant is licensed for `voice_bot`, a working config is seeded immediately: a sector-matched template if one exists (falls back to a generic template, then to bare defaults), greeting auto-filled with the tenant's real name, and IVR menu segments derived from what was actually licensed — Inquiry/FAQ self-service is always included, Complaint appears if `ticketing` is licensed, Sales appears if `sales` is licensed.

All three were live-tested against disposable test tenants/templates, not real client workspaces, with test data deleted afterward — except one self-caught slip: the agent-template assign endpoint was first tested against the real Haier Electronics tenant, briefly overwriting its greeting/tone/ language/system prompt with test values before the mistake was caught and the original values restored.

5. Mobile App Architecture

The field-staff mobile app (`packages/mobile`) is an Expo/React Native app used by sales and support staff working outside the office. It talks to the **exact same REST API** as the web app (same JWT auth, same endpoints) — there is no separate mobile backend.

5.1 Architecture Diagram

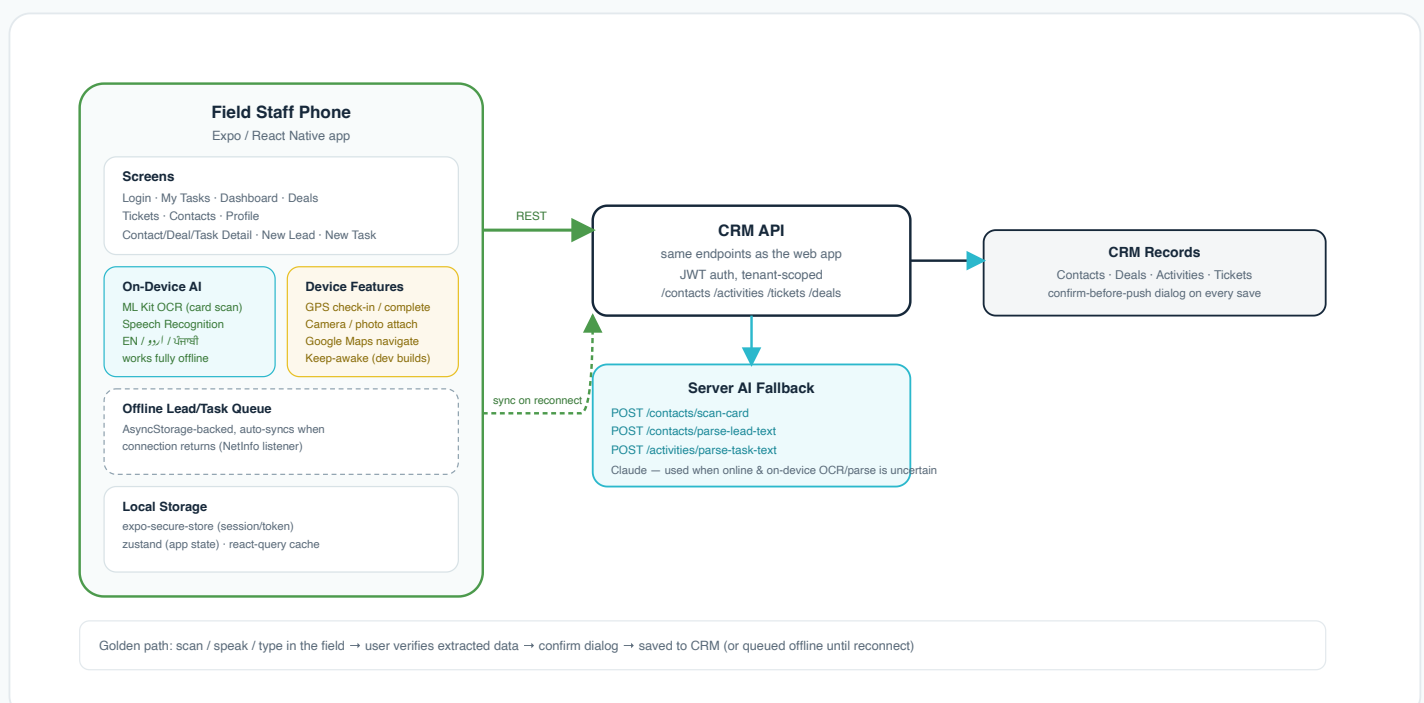


Fig 5.1 — Mobile app: same API contract as the web app, plus on-device AI and an offline queue for field conditions.

5.2 Screens & Functionality

SCREEN	PURPOSE
Login	Workspace slug + email + password (same JWT auth as web)
My Tasks	Landing screen — assigned activities/tasks, role-aware chips (sales → leads, support → tickets)
Dashboard	Stat cards + To-do/Done task cards (tappable → filters)
Deals	Pipeline list; Deal Detail includes photo attachments (camera/gallery)
Tickets / Contacts	Same records as the web CRM, filtered to the user's visibility scope
New Lead	Scan a visiting card (camera) or speak (voice capture) → extracted/verified fields → save as contact/lead
New Task	Voice capture → AI resolves relative dates/priority/type, auto-matches an existing contact by name
Task Detail	Call / Navigate (opens Google Maps) / GPS Check-in / Complete with outcome

5.3 Offline & On-Device AI

Works with no internet

- **Card scan:** on-device ML Kit text recognition (Android/iOS) extracts name/title/company/email/phone from a photo — no network call.
- **Voice capture:** on-device speech recognition, multilingual (English / Urdu / Punjabi), with an offline dictation parser as fallback.
- **Save:** if offline, the record is written to a local queue and auto-synced the moment connectivity returns (NetInfo listener).

Upgrades when online

- Card photo / raw dictation can be sent to the server, where Claude produces a cleaner structured extraction than the on-device pass.
- Every save — offline or online — shows a confirm dialog summarising what will be pushed to the CRM, so the field worker verifies AI-extracted data before it's committed.

6. Connecting Other Applications

This section is for integrating a customer's **other** systems — a softphone, an existing CRM, a BI tool, or an automation platform — with AmanahCX itself (as opposed to Section 4, which is about connecting Nadia to a different CRM).

6.1 REST & GraphQL API

All CRM data is reachable at `/api/v1/*` (REST) and via GraphQL (Mercurius). Both share the same auth, tenant resolution, and access-control middleware described in 3.3. This is the primary integration surface — the web app, mobile app, and Nadia's connector all use it.

6.2 Outbound Webhooks

A tenant can subscribe an external endpoint to CRM events and receive a signed POST whenever they fire:

```
POST /api/v1/webhooks
{
  "name": "My Integration",
  "url": "https://partner-system.example.com/hooks/amanahcx",
  "events": ["deal.won", "contact.created", "ticket.created", "voice.call_completed"]
}
```

Deliveries are HMAC-SHA256 signed; the receiver verifies with the shared secret. Failed deliveries land in a dead-letter queue with a replay endpoint (`GET /webhooks/dead-letter` , `POST /webhooks/dead-letter/:deliveryId/replay`) — useful for building a reliable sync against a customer's other system.

6.3 API Keys (Inbound)

For a third-party system to *call into* AmanahCX programmatically (rather than receive events from it), it authenticates with a scoped API key instead of a user JWT:

```
Authorization: ApiKey crm_live_XXXXXXXXXXXXXXXXXX
X-Tenant-Id: <tenant-uuid>
```

Keys are scoped per resource/action (e.g. `contacts:read` , `deals:write` , `voice:read`) — issue the narrowest scope the integration actually needs. Managed at `Settings → Integrations → API Keys` (tenant admin only).

6.4 Telephony / Softphone Integration

Two distinct patterns exist depending on what you're building:

- **Voice bot number (Nadia):** SIP trunk → LiveKit dispatch rule, as described in Section 4. This is for AI-handled inbound calls.
- **Human agent softphone:** the `connectors` module (`modules/connectors`) has provider adapters (Twilio, Vonage today) for agent-facing calling — inbound webhook normalisation, outbound call initiation, and call-completed events into the CRM's activity/ticket timeline. A softphone product integrates the same way a new telephony provider adapter would: implement the provider webhook contract and register it as a connector, rather than hand-building CRM-side call logging.

6.5 Common Integration Patterns

YOU'RE BUILDING...	USE...
A softphone for human agents	Telephony connector adapter (§6.4) + webhooks for call-completed events
Nadia talking to a different CRM	New <code>CRMConnector</code> implementation (§4.3/4.4) — no AmanahCX API involved at all
Syncing AmanahCX records into another system	Outbound webhooks (§6.2) for real-time push, or scoped API key polling (§6.3) for batch sync
Letting another system create/update CRM records	Scoped API key (§6.3) calling REST/GraphQL directly
An automation platform (Zapier-style)	Webhooks as the trigger source + API key as the action target — the same two primitives cover both directions

7. Data Model & Auth Essentials

Auth mechanisms

JWT	Human users (web + mobile). <code>Authorization: Bearer <token></code>
API Key	Programmatic/integration access, scoped per resource:action
Bearer secret	Service-to-service — e.g. Nadia → CRM uses <code>LIVEKIT_INGEST_SECRET</code>

Key data-model facts

- Every ticket links to a contact (auto-matched or auto-created on inbound).
- A `sales` ticket converts to a pipeline deal on acceptance (idempotent, `tickets.deal_id`).
- Per-channel communication consent (WhatsApp/SMS/Email) is append-only and enforced before outbound sends.
- SLA policies match by priority + channel/department/tags; escalation is multi-step with % thresholds.

8. Environments & Deployment

PIECE	HOSTING	DEPLOY TRIGGER
Frontend (packages/frontend)	Vercel	Auto-deploy on push to GitHub main
API (packages/api)	Railway	Auto-deploy on push to GitHub main
Database	Supabase (PostgreSQL, TLS pooler)	Migrations run as part of deploy
Nadia voice agent	Railway — a second, separate service in the same Railway project as the API, <code>rootDirectory=services/nadia-voice-agent</code>	Auto-deploy on push, but scoped : only rebuilds when

		files under that folder change
Mobile app	EAS build → Android APK direct-install; iOS via TestFlight (Apple Developer account required)	Manual build trigger (<code>eas build</code>) — not connected to GitHub auto-deploy at all

GitHub, Railway, Vercel, and Supabase are kept continuously in sync — there is no manual deploy step for the CRM web/API. Repo: `github.com/munirrazaa/AmanahCX` (main). See §9 for how these three build pipelines actually differ in practice, and §10 for what changes if you move off this managed stack.

9. Consolidated Build & Release Report

One GitHub repository feeds **three independent build pipelines**, plus a fourth path (mobile) that isn't connected to GitHub auto-deploy at all. Understanding which push triggers which build — and which one doesn't auto-trigger anything — matters for anyone shipping changes: pushing to `main` silently ships to two production services at once (frontend + API) but only touches Nadia if you changed her folder, and never touches the mobile app.

9.1 Three Pipelines, One Repo



Fig 9.1 — one repo, three auto-deploy targets, and one manually-triggered mobile build path that isn't wired to any of them.

9.2 CRM Web + API Builds (GitHub → Vercel + Railway)

A representative sample — the full history (60+ entries) lives in `docs/CHANGE_LOG.md` , not reproduced in full here.

DATE	PUSH	WHAT SHIPPED
------	------	--------------

2026-06-29	Push 5 of 5	Policy Admin governance role — independent SLA-policy write access, ranked above Manager
2026-07-01	—	SLA governance guardrail (mandatory reason for priority changes) + reassignment audit trail
2026-07-10	Critical fix	Tenant data-isolation fix — restricted <code>crm_app</code> DB role (migration 048) + Vercel reconnected to the correct repo for auto-deploy
2026-07-13	Push 2 of 2	Login "spinner forever" root-cause fixed + Voice Bot admin screen added
2026-07-14	Push 3	SLA Save bug fixed — was blocked by four separate stacked causes
2026-07-15	Push 15	Super Admin can centrally hold Voice Bot + Integrations config per tenant (Phase 1)
2026-07-16	Push 2	Manager Field-Team View built
2026-07-17	Push 3	Full 5-role audit across all 8 tenants; 4 crash fixes; Super Admin sidebar nav fix

9.3 Nadia Voice Agent Builds (GitHub → Railway, scoped path)

Nadia's Railway service builds from a `python:3.11-slim` Dockerfile that also installs `ffmpeg` (required for the speaking-rate time-stretch, §4.2) and pre-downloads the Whisper model at build time so the first live call isn't stuck on a cold download.

DATE	PUSH	WHAT SHIPPED
2026-07-13	Initial build	Nadia built — self-hosted LiveKit agent (Whisper STT + Uplift AI TTS + gpt-4o-mini), Pakistani-market alternative to Retell/Vapi
2026-07-14	Push 1 of 2	First real production deployment — 2nd Railway service, Dockerfile, <code>agent_name</code> mismatch bug fixed (calls were never being picked up), scoped auto-deploy wired up
2026-07-14	Push 6	Production call-quality — two root causes fixed
2026-07-14	Push 8	SIP wiring audit — 3 real phone-call bugs fixed + <code>setup_sip.py</code> automation added
2026-07-14	Push 9	Silence handling — "are you still there?" nudge sequence
2026-07-14	Push 10	Audio call recording with spoken consent notice

2026-07-14	Push 13	Knowledge Base tool — answers general questions before raising a ticket
2026-07-17	Push 2	Hold-message during ticket creation + per-minute voice bot cost tracking

9.4 Mobile App Builds (EAS — not part of GitHub auto-deploy)

Mobile releases are a deliberate, manual step (`eas build --profile <profile> --platform android`) — there is no equivalent of Vercel/Railway's "push and it's live" for the app. Whatever is on the developer's disk at build time is what ships; a build cut today does not automatically pick up an API change merged tomorrow.

DATE	BUILD	WHAT SHIPPED
2026-07-06	Dev build config	Offline on-device OCR scanner + voice lead/task capture built; Vivid rebrand applied; first <code>eas.json</code> profile created
2026-07-07	APK #2	<code>eas build --non-interactive</code> — deal photo attachments, login-path fix; distributed as a direct-install APK
2026-07-11	Code-level (no new APK)	Field-visit flow (GPS check-in/complete) recovered from a stash conflict and re-tested against the cloud database
2026-07-12	Live phone test	First live test of the production APK on a real Android phone; 3 field-capture bugs found and fixed

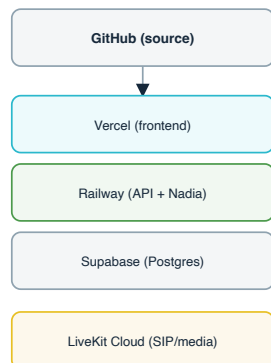
Full, unabridged history for all three pipelines: `docs/CHANGE_LOG.md` . This report is a curated sample to show the pattern and cadence, not a replacement for it.

10. Self-Hosted Migration (VM + Domain)

This section is a **ready-to-execute runbook** for moving off Railway/ Vercel/Supabase onto infrastructure you control — a single VMware VM reachable by a domain name. Hand this to whoever provisions the VM (ops/infra) — steps 1–11 below are written to be followed in order, with the one non-negotiable called out explicitly at step 4 (recreating the restricted `crm_app` role so Row-Level Security keeps working).

10.1 Target Architecture Diagram

TODAY — MANAGED SERVICES



→

TARGET — ONE VM + YOUR DOMAIN

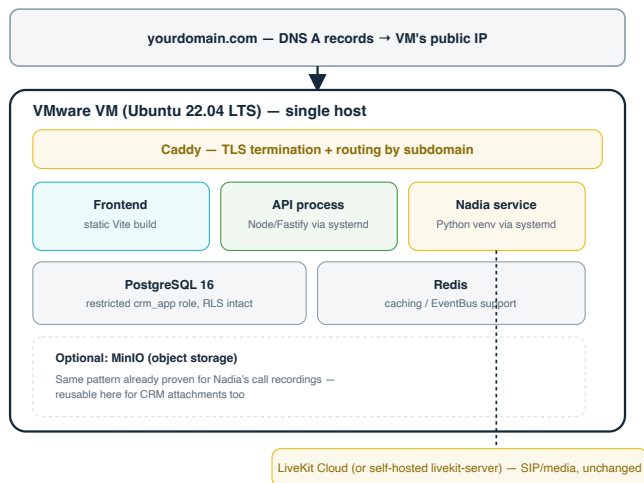


Fig 10.1 — collapsing four managed services into one VM behind your own domain; LiveKit's SIP/media plane can stay external or move in too.

10.2 Migration Runbook

- 1 **Provision the VM.** Ubuntu 22.04 LTS, reserved/static internal IP, a public IP (or NAT port-forward for 80/443). Sizing: 4 vCPU / 8GB RAM covers frontend + API + Postgres comfortably; if Nadia's self-hosted Whisper STT also runs on this box, budget significantly more CPU — Whisper is what maxed out Railway's small hosts in production (§9.3), which is why `STT_PROVIDER=openai` (hosted transcription) exists as a fallback for constrained hardware.
- 2 **Install the base stack.** Node.js 20 LTS, Python 3.11, PostgreSQL 16, Redis, Caddy (automatic HTTPS — the same pattern already used for Nadia's MinIO recording storage), and **ffmpeg** (Nadia's speaking-rate control depends on it, §4.2 — easy to miss since nothing fails loudly without it until a tenant sets a non-default speaking rate).
- 3 **Clone the repo.** GitHub stays the source of truth — `git clone github.com/munirrazaa/AmanahCX` onto the VM.
- 4 **Migrate the database — the step that must not be shortcut.** `pg_dump` the Supabase database → restore into local Postgres → recreate the restricted `crm_app` role with `NOSUPERUSER NOBYPASSRLS` and only the grants it needs (§3.2 — either attribute silently disables every Row-Level Security policy). Point `DATABASE_URL` at `crm_app@localhost`, never at a superuser connection.
- 5 **Build and run the API.** `npm install && npm run build` in `packages/api` → run the compiled `dist/` under a systemd service on an internal port, proxied by Caddy at `api.yourdomain.com`.
- 6 **Build and serve the frontend.** `npm run build` (Vite) in `packages/frontend` → the static `dist/` files are served directly by Caddy at `app.yourdomain.com` — no Node process needed, same as Vercel today, just served locally.
- 7 **Deploy Nadia.** Python venv, `pip install -r requirements.txt`, confirm `ffmpeg` is on PATH, set `CRM_API_BASE_URL` to the new internal API address, run `src/agent.py` as a systemd service with `agent_name=nadia` set (§9.3 — this exact value mismatch broke production once already). `LIVEKIT_URL` / `API_KEY` / `API_SECRET` keep pointing at LiveKit Cloud unless you also self-host `livekit-server` on this VM.
- 8 **Point the domain at the VM.** Create A records for `app.yourdomain.com` and `api.yourdomain.com` → the VM's public IP. Caddy auto-provisions Let's Encrypt certificates the moment DNS resolves — no manual certificate step.
- 9 **Lock it down.** Only open 80/443 publicly; keep Postgres bound to localhost (never expose 5432); SSH via key-based auth only; keep `crm_app` non-superuser as the one non-negotiable carried over from today's setup.
- 10 **Replace the "push and it's live" convenience.** Railway/Vercel's auto-deploy doesn't exist for free on a bare VM — the simplest equivalent is a small webhook receiver (or a GitHub Actions self-hosted runner) that runs `git pull && npm run build && systemctl restart api` (and rebuilds the frontend static files) on every push to main. This is the one piece of automation the team now owns and must consciously rebuild.

11 Back up. Nightly `pg_dump` cron to storage off the VM, plus a VMware-level snapshot before any major change — a safety net most managed PaaS platforms don't hand you directly.

10.3 What You Gain, What You Now Own

CONCERN	TODAY (MANAGED)	SELF-HOSTED VM
TLS certificates	Handled automatically by Vercel/Railway	Caddy auto-provisions via Let's Encrypt — still automatic, but it's now your Caddy config
OS & runtime patching	Not your concern	Your team's responsibility (apt updates, Node/Python version bumps)
Auto-deploy on push	Free, built-in	You build and maintain the pull-and-restart script (§10.2 step 10)
Database backups	Supabase-managed	Your cron job + your retention policy
Scaling	Usage-based, elastic	Fixed VM capacity — scaling means resizing the VM or adding another one
Cost model	Usage-based billing across 3 vendors	One fixed VM cost, no per-request/per-build billing
Data residency / control	Data lives in Vercel/Railway/Supabase's infrastructure	Full control — everything sits on hardware you (or your VMware host) own
Incident recovery	Vendor status pages, vendor SLAs	VM snapshots give instant rollback, but you're the one restoring it at 2am

11. Repository Map

```
AmanahCX/solution/
├── packages/
│   ├── frontend/      React + Vite SPA – tenant app + super-admin console
│   ├── api/           Fastify REST + GraphQL server, all route handlers
│   ├── core/          DatabaseClient, EventBus, ModuleRegistry, TenantService
│   ├── shared/        Cross-package types, sector config, contracts
│   └── mobile/         Expo/React Native field-staff app
├── modules/
│   ├── contacts/ companies/ deals/ activities/
│   ├── ticketing/ sales/ analytics/ voice/ voice-module/
│   └── connectors/     Twilio/Vonage telephony provider adapters
├── services/
│   └── nadia-voice-agent/ Independent Python LiveKit voice service
```

```

├── src/
│   ├── agent.py          entrypoint, pipeline, tools, silence/capacity guards
│   ├── config.py         per-tenant AgentSettings + system-prompt builder
│   ├── crm_client.py     CRMConnector contract + AmanahCXConnector
│   ├── setup_sip.py      one-time SIP trunk + dispatch rule provisioning
│   ├── stt_whisper.py    self-hosted Whisper STT wrapper
│   └── recording.py      call recording + consent line
├── docs/                Product docs – MASTER_PRODUCT_DOCUMENT.md,
                        CHANGE_LOG.md, OPERATIONS_GUIDE.md, this file
└── infra/               Docker/K8s manifests (legacy – current hosting is
                        Vercel/Railway/Supabase, see §8)

```

Adding a new business module to the CRM: implement the `CRMModule` interface (`onLoad` , `registerRoutes`) and register it in `packages/api/src/server.ts` — the ModuleRegistry handles load order and route mounting.

12. Glossary

TERM	MEANING
Tenant	One isolated customer organisation on the shared AmanahCX platform
RLS	Row-Level Security — Postgres feature enforcing tenant isolation at the database layer
Nadia	AmanahCX's self-hosted AI voice agent (LiveKit-based)
LiveKit	Real-time media/SFU platform Nadia runs on; also brokers SIP trunking and browser WebRTC test calls
SIP trunk	The telecom connection that delivers real phone calls into LiveKit
Dispatch rule	LiveKit config that routes an inbound SIP call into a room and starts the Nadia agent in it
Connector (Nadia)	The 6-method adapter class that lets Nadia talk to any CRM (§4.3)
Connector (CRM)	A telephony/messaging provider adapter (Twilio, Vonage) inside <code>modules/connectors</code> — different meaning, same word
Entitlement	What a tenant has actually licensed (modules/features) — Layer 1 of access control
Policy Admin	Governance-only role with sole write access to SLA policies, ranked above Manager
TAT / SLA	Turn-Around-Time / Service-Level-Agreement — the deadline clocks on tickets

